

TaskFlow

Project & Team Task Management Platform

Feature Completion Report

Submitted for — Intern Full Stack Developer Position

CyphLab (Private) Limited

Candidate: Ahmad Hassan

Position: Intern Full Stack Developer

Date: July 16, 2026

Repository: <https://github.com/Ahmadhassan011/TaskFlow>

This report documents the completion status of all features required by the CyphLab practical assignment.

Contents

1	Executive Summary	2
2	Core Features	2
2.1	Administrator Capabilities	2
2.2	Project Manager Capabilities	2
2.3	Team Member Capabilities	3
3	Technical Requirements	3
4	Additional Features (Beyond Requirements)	4
5	Database Design	4
6	Security	5
7	Testing	5
8	CI/CD Pipeline	5
9	API Endpoints Summary	6
10	Submission Artifacts	6
11	AI Tools Disclosure	7
12	Conclusion	7

Executive Summary

TaskFlow is a full-stack, multi-tenant project and task management platform built with Next.js 16 (App Router) on the frontend, Express.js with TypeScript on the backend, and PostgreSQL via Prisma ORM.

All core features required by the assignment have been fully implemented, along with several additional features that strengthen the submission:

- Complete role-based access control with 5 roles and 32 granular permissions
- Full multi-tenancy with workspace isolation and invitation flow
- Kanban board with drag-and-drop task management
- JWT authentication with email verification and password reset
- Comprehensive audit trail logging every authorization decision
- Interactive Swagger API documentation and Postman collection
- GitHub Actions CI/CD pipeline with lint, build, and test stages
- 68 passing unit tests across 8 test suites

Core Features

Administrator Capabilities

Capability	Status	Details
Manage users (invite, edit, deactivate)	Complete	Invite by email, edit name/email/role, deactivate/activate, accept invitations
Manage roles	Complete	5-tier hierarchy: OWNER $\supset$ ADMIN $\supset$ MANAGER $\supset$ MEMBER $\supset$ GUEST
Manage projects	Complete	Full CRUD, member assignment, label management
System access & workspace settings	Complete	Workspace name update, profile management
View audit log	Complete	Paginated, filterable by ALLOW/DENY, shows actor, role, IP, reason
Manage invitations	Complete	Send, list, revoke invitations; token-based accept flow

Project Manager Capabilities

Capability	Status	Details
Create projects	Complete	Name, description, status, color, deadline
Edit / delete projects	Complete	Full CRUD via dropdown actions
Assign team members to projects	Complete	Role-based assignment per project (MANAGER, MEMBER, GUEST)

Capability	Status	Details
Remove / update project members	Complete	MemberManager component with role selector
Manage project labels	Complete	Color-coded labels, add / remove from tasks
Create and manage tasks	Complete	Full CRUD, assignee, priority, deadline, description
Share resources	Complete	Per-resource access (VIEW / EDIT / MANAGE) via ResourceShare

Team Member Capabilities

Capability	Status	Details
View assigned projects	Complete	Auto-filtered by role scope (<code>read:assigned</code>)
View assigned tasks	Complete	Kanban board with 4 columns: TODO, IN_PROGRESS, IN_REVIEW, DONE
Update task status	Complete	Drag-and-drop via @dnd-kit; linear progression enforced for MEMBER/MANAGER
Add comments	Complete	Threaded comments with user avatars
Manage subtasks	Complete	Toggle, add, delete subtasks with progress tracking
Upload attachments	Complete	File upload via Multer, list and delete
View notifications	Complete	Real-time bell icon with read / unread state
View activity feed	Complete	User-level and entity-level activity logging
View dashboard stats	Complete	Scoped stats, status distribution, upcoming deadlines, workload

Technical Requirements

Requirement	Implementation	Status
Frontend: Next.js	Next.js 16.2.10, App Router, React 19, Tailwind v4, shadcn	Complete
Backend: Node.js	Express 4.21.1, TypeScript, Zod validation, Multer	Complete
Database: PostgreSQL	Prisma 5.22.0, 19 models, 25+ indexes	Complete
Secure authentication	JWT (access 15 min + refresh 7 d), bcrypt, email verify, password reset	Complete
RBAC	5 roles, 32 permissions, ReBAC fallback, audit logging	Complete
RESTful API	11 route modules, nested resources, pagination, filtering	Complete

Requirement	Implementation	Status
Database relationships	19 models, proper cascades, unique constraints	Complete
Responsive UI	Tailwind responsive, mobile sidebar, FAB, responsive grids	Complete
Git version control	GitHub repository with meaningful commit history	Complete
CI/CD pipeline	GitHub Actions: lint, build, test for frontend & backend	Complete

Additional Features (Beyond Requirements)

Feature	Description
Kanban Board	Drag-and-drop task status management with @dnd-kit, horizontal scroll on mobile
Task Comments	Threaded comments on tasks with user avatars and timestamps
File Attachments	Upload, list, and delete file attachments on tasks (Multer)
Subtasks	Toggle-able subtask lists with completion progress tracking
Activity Feed	Entity-level and user-level activity logging with timestamped entries
Notifications	Real-time notification bell with read / unread state and mark-all-read
Multi-tenancy	Workspace isolation, invitation system, workspace switcher
Audit Trail	Every authorization decision logged with actor, role, action, resource, IP
Resource Sharing	Cross-user resource access via ResourceShare (ReBAC model)
Project Labels	Color-coded labels for task organization and filtering
Dark Mode	System-preference detection + manual toggle with local-storage persistence
Swagger API Docs	Interactive OpenAPI 3.0.3 documentation served at <code>/api/docs</code>
Postman Collection	Importable collection with all endpoints and auto-capture scripts

Database Design

The database schema consists of 19 models with comprehensive relationships and 25+ indexes.

Model	Key Relationships
User	Tenant (owner), TenantMembership, Project, Task, Comment, RefreshToken
Tenant	TenantMembership, Invitation, ResourceShare, AuditLog, Project, Task, Label, ActivityLog, Notification
Project	ProjectMember (with role), Task, Label

Model	Key Relationships
Task	Subtask, Comment, TaskLabel, Attachment, ActivityLog
Label	TaskLabel (many-to-many with Task)
Invitation	Token-based, single-use, expiry
RefreshToken	DB-persisted, device fingerprinted
AuditLog	Every auth decision (ALLOW/DENY) with full context
ResourceShare	Per-resource access (VIEW / EDIT / MANAGE)
ActivityLog	Entity-level activity tracking
Notification	User-scoped, read / unread state

Security

- **JWT Authentication** – Access tokens (15 min) + refresh tokens (7 d), bcrypt password hashing (12 rounds)
- **Refresh Token Rotation** – DB-persisted tokens, invalidated on use
- **Email Verification** – Token-based, 24 h expiry, hashed in DB
- **Password Reset** – Token-based, 1 h expiry, hashed in DB
- **Rate Limiting** – express-rate-limit on auth endpoints
- **CORS** – Configurable origin whitelist
- **RBAC + ReBAC** – Role permissions checked first; per-resource shares can elevate access
- **Audit Logging** – Every authorization decision logged with actor, role, action, resource, IP

Testing

68 unit tests across 8 test suites using Vitest.

Test Suite	Tests	Status
Auth Service	11	Complete
User Service	10	Complete
Project Service	9	Complete
Task Service	15	Complete
Comment Service	10	Complete
Label Service	7	Complete
Dashboard Service	3	Complete
Health Check	3	Complete
Total	68	All Passing

CI/CD Pipeline

GitHub Actions workflow (`.github/workflows/ci.yml`) runs on every push and PR to main.

Step	Backend Job	Frontend Job
1	Checkout code	Checkout code
2	Setup Node.js 20 with npm cache	Setup Node.js 20 with npm cache
3	Install dependencies (npm ci)	Install dependencies (npm ci)
4	Lint (npm run lint)	Lint (npm run lint)
5	Build TypeScript (npm run build)	Build Next.js (npm run build)
6	Generate Prisma client	—
7	Run unit tests (npm run test:run)	—

Both jobs run in parallel for faster feedback.

API Endpoints Summary

Module	Count	Key Operations
Auth	8	Register, login, refresh, profile, password, forgot-password, verify-email, switch-tenant
Users	4	List, get, update, deactivate (admin-only)
Tenants	8	Get current, update, members, invites (send/list/accept/revoke), audit-logs, shares
Projects	8	CRUD, list members, add/update/remove members
Tasks	8	CRUD, status change, subtasks (list/create/toggle), labels (list/add)
Labels	4	List, create, update, delete (project-scoped)
Comments	4	List, create, update, delete (task-scoped)
Dashboard	1	Stats (task counts, status distribution, deadlines, workload)
Activity	2	User activity feed, entity activity feed
Notifications	3	List, mark read, mark all read
Attachments	3	Upload, list, delete
Health	1	Health check
Total	54	

Submission Artifacts

Artifact	Location	Status
GitHub repository	github.com/Ahmadhassan011/TaskFlow	Complete
Screen recording	To be recorded	Pending
README	README.md	Complete
.env.example	backend/.env.example, frontend/.env.example	Complete

Artifact	Location	Status
ERD	docs/diagrams/erd.png	Complete
Architecture diagram	docs/diagrams/architecture_diagram.png	Complete
Sequence diagram	docs/diagrams/sequence_diagram.png	Complete
Activity diagram	docs/diagrams/activity_diagram.png	Complete
API documentation	docs/api-reference.md + Swagger UI	Complete
Postman collection	docs/postman-collection.json	Complete
Feature completion report	docs/feature-completion-report.pdf	Complete
CI/CD workflow	.github/workflows/ci.yml	Complete

AI Tools Disclosure

Tool	Usage
GitHub Copilot	Code autocomplete, boilerplate generation, debugging assistance
ChatGPT	Architecture planning, Prisma schema design, API documentation drafting
Cursor	Code generation, refactoring suggestions, error resolution

All AI-assisted code was reviewed, tested, and adapted to fit the project's specific requirements and coding conventions. No code was copied verbatim without understanding and verification.

Conclusion

TaskFlow delivers a production-quality, multi-tenant project management platform that meets or exceeds every requirement specified in the CyphLab practical assignment:

- All three user roles (Admin, Project Manager, Team Member) are fully functional
- The technical stack (Next.js, Express, PostgreSQL, Prisma) is modern and well-structured
- Security is production-grade with JWT auth, 5-role RBAC, and comprehensive audit logging
- The codebase includes 68 passing unit tests and a working CI/CD pipeline
- Documentation is thorough: README, API reference, Swagger UI, Postman collection, diagrams, and this report

The platform is ready for deployment and demonstrates strong full-stack development capabilities.